LINUX FOR AI-ML SYSTEMS · MODULE 2 OF 3

Feed the Model

Most training jobs that die at 97% died before they started. **Will it fit? Can it read? Is it secret?**

INSTRUCTOR

Emmanuel Naweji

RUNTIME

~45 min · 4 sections · 1 lab

COURSE REPO

github.com/Here2ServeU/linux-for-ai-ml

THE ONE IDEA FOR THIS MODULE

Three questions before you spend one GPU hour

Most jobs that die at 97% died before they started. Ask these first.

01

Will it fit?

Is there room in the scratch space for every checkpoint this run will write?

02

Can it read?

Can the job open its training data? One missing r and it crashes on line one.

03

Is it secret?

Is the access token out of the config files, in a private .env file only you can read?

Memory hook: Will it fit? Can it read? Is it secret? Say it before you press Enter on any long job.

SECTIONS AT A GLANCE

Four sections, one lab, one checklist

01 **Will It Fit?** ~8 min

df -h + du -sh before every run

02 **Who Can Read It?** ~10 min

ls -l, chmod, chown

03 **Keep Secrets Secret** ~10 min

.env, chmod 600, never in Git

04 **Build It: The Pre-Flight Script** ~12 min

One script says GO / NO-GO

The lab runs through all four: you build a training job, find its three problems, fix them, and score 3 / 3.

READ THIS FIRST

How to type the commands in this course

If you've never used a terminal, this slide is for you.

```
$ cd ~/ai-job   # go into the ai-job folder
```

`$`	The prompt. Linux shows it. Don't type it.
`cd ~/ai-job`	Type exactly this, then press Enter.
`# go into…`	A comment: a note for you. Don't type it.
`~`	Your home folder, e.g. /home/ubuntu

One line at a time

Type one line, press Enter, and wait for the prompt to come back before typing the next line.

Check the green box

After each step, compare your screen with YOU SHOULD SEE. Match? Move on. Different? See the Stuck? slide near the end.

Update the course repo and check your machine

```
$ cd ~/linux-for-ai-ml   # go to the course repo
$ git pull   # get the newest lab files
$ bash setup/doctor.sh   # check Linux, Python, Git, disk
```

YOU SHOULD SEE

```
Every line says OK
```

Using Codespaces?

The repo lives in a different place. Use: cd /workspaces/linux-for-ai-ml

"No such file or directory"?

You don't have the repo yet. Do Step 1 of Module 1 first, then come back.

Same machine as Module 1

Codespaces, WSL, Multipass or Linux. Nothing new to install.

THE JOB THAT DIED AT 97%

Last night: almost six good hours, one bad minute

This is the real log from the lab job you're about to build: logs/run-07.log

01:00

run-07 starts

dataset opened, writing checkpoints to scratch/run-07

06:41

step 9500 · 97%

checkpoint saved

06:44

step 9600 · 97%

checkpoint saved

06:48

ERROR

No space left on device: scratch/run-07/step-9700.pt

5h 48m

of training lost. The code was fine. The GPU was fine. The scratch space was full of an old run's temp files nobody deleted. Run-08 is next, and it will fail too, unless you check first.

Build ai-job, then read its briefing

```
$ cd ~/linux-for-ai-ml/modules/module-2/scripts
$ python3 make_job.py   # builds ~/ai-job
$ cd ~/ai-job   # walk into the job
$ cat README-FIRST.txt   # print the briefing
```

YOU SHOULD SEE

```
Built your training job at /home/you/ai-job
Next:  cd /home/you/ai-job && cat README-FIRST.txt
Welcome to ai-job. Last night's run died at 97%…
```



"already exists"?

You built it before. Rebuild it fresh:
python3 make_job.py --reset



Your folder name

Your home folder replaces /home/you, e.g. /home/ubuntu. The job is always ~/ai-job.

What's inside ~/ai-job

```
$ ls
```

YOU SHOULD SEE

```
README-FIRST.txt  check_answers.sh
configs  datasets  logs
preflight.sh  scratch
```

datasets/

The training data: train.jsonl and eval.jsonl

scratch/

Checkpoints go here. 40 MB quota

configs/

Settings for the run: train.yaml, eval.yaml

logs/

Last night's log: run-07.log

preflight.sh

The pre-flight check: says GO or NO-GO

check_answers.sh

Scores your three answers

Sizes are scaled: 1 MB here stands for 1 GB on a real server. The token in this job is fake.

Run the pre-flight: it says that something is wrong, not why

```
$ bash preflight.sh   # run from inside ~/ai-job
```

YOU SHOULD SEE · NO-GO IS EXPECTED

```
[FAIL] Disk: scratch has 7 MB left of its 40 MB quota, the run needs 20 MB
[FAIL] Permissions: datasets/train.jsonl is --w-------, the job can't read it
[OK]   Permissions: datasets/eval.jsonl is readable (-rw-r--r--)
[FAIL] Secrets: a token is pasted into configs/train.yaml
NO-GO: 3 check(s) failed. Fix them before you start the run.
```

Mission 1 · Disk

what is filling scratch?

Mission 2 · Permissions

which file can't be read?

Mission 3 · Secrets

which file has a token in it?

01

Will It Fit?

df -h and du -sh before every download or training run.

~8 min

Two different kinds of "full"

The disk can have room while your job's quota is full. Check both.



`df -h .`

Free space on the disk you're standing on. Look at Avail and Use%. The dot means "this folder".



`du -sh scratch/* | sort -h`

Who is using the space inside scratch. Smallest first, so the biggest is on the bottom line. You met this in Module 1.



`df -i`

Inodes: how many files a disk can hold. Millions of tiny images can fill it while GBs still look free.



Our job: the disk has plenty of room, but the job may only use 40 MB of scratch. That's the full one. Like a parking garage with space, but your 40 reserved spots are taken.

Mission 1: what is filling the scratch space?

```
$ cd ~/ai-job   # make sure you're in the job
$ grep -rn ERROR logs   # what killed last night's run?
$ df -h .   # is the disk full?
$ du -sh scratch/* | sort -h   # who fills scratch?
```

YOU SHOULD SEE

```
logs/run-07.log:6:… ERROR No space left on device
(df) Avail shows GBs free: the disk is fine
516K    scratch/cache
2.1M    scratch/run-07
31M     scratch/run-03-tmp
```

31 MB

Read the bottom line

run-03-tmp: temp shards from an old run, 31 MB of the 40 MB quota. Run-08 needs 20 MB.

Your answer

Mission 1 = scratch/run-03-tmp
Write it down: the grader asks for it in Step 6.

Check nothing uses it, then delete it

```
$ grep -rn run-03 configs   # does any config use it?
$ rm -r scratch/run-03-tmp   # delete the folder
$ bash preflight.sh   # check again
```

YOU SHOULD SEE

```
(grep prints nothing: no config uses run-03)
[OK]   Disk: scratch has 37 MB left, the run needs 20 MB
[FAIL] Permissions: … (still 2 to go)
```

No output is good news

grep printing nothing means no config points at run-03-tmp. Safe to remove.

rm has no undo and no trash can. Measure, check, then delete exactly the folder you named.

Rule for real jobs: checkpoint math

One checkpoint's size × how many you keep + 20% headroom. 10 GB × 10 kept = 100 GB, plus 20% = 120 GB. Doesn't fit? Keep fewer.

02

Who Can Read It?

Owners, groups and rwx: ls -l, chmod, chown.

~10 min

Ten characters tell you who can do what

```
$ ls -l datasets
```

```
-rw-r--r--  1 you you   614400 …  eval.jsonl
```

permissions · owner · group · size · name

-	rw-	r--	r--
TYPE	OWNER	GROUP	OTHERS
A file. d means a folder.	you can read and write.	your team can read.	everyone else can read.

r = read · w = write · x = run (execute) · - = not allowed. A hotel key card opens your room, not every room.

Numbers make it fast: r = 4, w = 2, x = 1

644	`rw- r-- r--`	Owner reads + writes, everyone else reads. Data files.
600	`rw- --- ---`	Only you. Use it for secrets like .env.
750	`rwx r-x ---`	Owner does all, group reads + runs, others nothing.
777	`rwx rwx rwx`	Everyone can do everything. Never. It "fixes" errors by removing all protection.



Add them up

rw- = 4 + 2 + 0 = 6
r-- = 4 + 0 + 0 = 4
So rw-r--r-- = 644

`chmod 644 f`	who can read / write
`sudo chown`	change the owner

Mission 2: which file can the job not read?

```
$ ls -l datasets    # look for a missing r
```

YOU SHOULD SEE

```
-rw-r--r-- 1 you you  614400 … eval.jsonl
--w------- 1 you you 6291456 … train.jsonl
```

```
$ chmod 644 datasets/train.jsonl    # owner rw, others r
$ ls -l datasets/train.jsonl    # check it
$ bash preflight.sh    # check again
```

YOU SHOULD SEE

```
-rw-r--r-- … datasets/train.jsonl
[OK]   Permissions: datasets/train.jsonl is readable
```



--w------- means

The owner can write, but nobody can read. The job would open the file and crash.

Your answer

Mission 2 = datasets/train.jsonl

Shared server?

If ls -l shows another owner: sudo chown you:you file

03

Keep Secrets Secret

API keys and tokens: environment variables, .env, chmod 600, never in Git.

~10 min

Same token. Two very different risks.

IN A CONFIG FILE

`hf_token: hf_LABfake…`

Configs get shared, copied into chat and committed to Git. A token in a config is a token on the internet.

IN .env (OR THE ENVIRONMENT)

`HF_TOKEN=hf_LABfake…`

.env is a hidden file next to the job. .gitignore keeps it out of Git, and chmod 600 means only you can read it.

1 · Move the token into .env

2 · chmod 600 .env

3 · .env is listed in .gitignore

Environment variables in four moves

A named value your shell gives every program it starts. Values here are fake, so it's safe.

```
$ export DEMO=hi   # set it (nothing prints: normal)
$ echo $DEMO   # show it
$ unset DEMO   # remove it
$ echo $DEMO   # empty line now
```

YOU SHOULD SEE

```
hi
(an empty line)
```

`export NAME=value`	set it for this shell
`echo $NAME`	print it ($ reads it)
`unset NAME`	remove it
`${NAME:0:6}`	only the first 6 characters



Never print a whole real token on screen or in a recording.

Mission 3: which file has a secret pasted into it?

```
$ grep -rn token configs   # search every config
```

YOU SHOULD SEE

```
configs/train.yaml:9:hf_token: hf_LABfakeTokenNotReal0000000000
```

`configs/train.yaml`	the file
`:9:`	line 9
`hf_token: hf_…`	the pasted token. hf_ = a Hugging Face token

Your answer

Mission 3 = configs/train.yaml

Fake token, so it's safe to see. With a real one, stop here and revoke it.

Move the token into .env, then lock it

```
$ grep hf_token configs/train.yaml | sed 's/hf_token: /HF_TOKEN=/' > .env
$ sed -i '/^hf_token:/d' configs/train.yaml   # delete the line
$ bash preflight.sh   # one more FAIL: that's expected
```

YOU SHOULD SEE · NOT DONE YET

```
[FAIL] Secrets: .env is -rw-r--r--, so other users can read it
         lock it down:  chmod 600 .env
```

```
$ chmod 600 .env   # only you can read it
$ ls -l .env   # check it
$ cat .gitignore   # .env is listed: Git ignores it
```

YOU SHOULD SEE

```
-rw------- … .env
.env
scratch/
```

`grep hf_token …`	find the token line
`| sed 's/A/B/'`	swap A for B
`> .env`	save into .env
`sed -i '/…/d'`	delete that line in place

04

Build It: The Pre-Flight Script

One script that says GO / NO-GO before a job starts.

~12 min

Pre-flight again, then check your answers

```
$ bash preflight.sh
$ bash check_answers.sh scratch/run-03-tmp datasets/train.jsonl configs/train.yaml
```

YOU SHOULD SEE

```
[OK]   Disk: scratch has 37 MB left, the run needs 20 MB
[OK]   Permissions: datasets/train.jsonl is readable (-rw-r--r--)
[OK]   Permissions: datasets/eval.jsonl is readable (-rw-r--r--)
[OK]   Secrets: no tokens in configs, HF_TOKEN comes from .env (600)
GO: all pre-flight checks passed. Safe to start the run.
Score: 3 / 3
Fix check: preflight.sh says GO. Run-08 is safe to start.
You just stopped a training run from dying at 97%.
```



[MISS]? Check the spelling of that path; the hint next to it shows which command finds it. Still NO-GO? Run bash preflight.sh and fix the FAIL it names.

Read the pre-flight script: ~70 lines of plain Bash

Every check uses a command from this module.

```
$ less preflight.sh   # arrows to scroll, q to quit
```

1 · Disk df · du

Is there room on the disk, and inside the 40 MB scratch quota?

2 · Permissions stat

Is the 2nd character of each data file an r?

3 · Secrets grep

Any hf_ or sk- token in configs? Is .env 600?

Any FAIL means NO-GO, and the script exits with an error, so you can run it automatically before every job.

Make it yours, then practice again

Challenge: add a fourth check

Fail if scratch/ holds any *.tmp file older than 7 days. The command you need:

```
$ find scratch -name '*.tmp' -mtime +7
```

Practice again: NO-GO to GO in 5 min

Rebuild the broken job, fix all three from memory.

```
$ cd ~/linux-for-ai-ml/modules/module-2/scripts
$ python3 make_job.py --reset
```

DONE? CLEAN UP

```
$ cd ~/linux-for-ai-ml/modules/module-2/scripts
$ python3 make_job.py --clean   # delete ~/ai-job
$ unset HF_TOKEN   # if you exported one
```

YOU SHOULD SEE

```
Removed /home/you/ai-job.
Disk reclaimed.
```

STUCK?

If your screen doesn't match, check here

`make_job.py: "already exists"`	You built it before: python3 make_job.py --reset
`"No configs/train.yaml here"`	Wrong folder: cd ~/ai-job then run it again
`cd ~/ai-job: "No such file or directory"`	Not built yet: Lab Step 2 (python3 make_job.py)
`cd ~/linux-for-ai-ml: "No such file…"`	Codespaces: cd /workspaces/linux-for-ai-ml · else do Module 1 Step 1
`python3 train.py: "can't open file"`	This lab has no train.py. The check is: bash preflight.sh
`check_answers.sh shows [MISS]`	A typo in a path. The hint next to it names the command that finds it
`Typed something wrong and stuck?`	Press Ctrl+C to cancel, then type the line again

RECAP

Three questions to carry into Module 3



Will it fit?

`df -h .` · `du -sh scratch/* | sort -h`

Measure, check nothing uses it, then rm.



Can it read?

`ls -l` · `chmod 644 file`

644 for data, 600 for secrets, never 777.



Is it secret?

`grep -rn token configs` · `chmod 600 .env`

Tokens live in .env or the environment, never in configs or Git.

SELF-CHECK · FROM MEMORY

No scrolling back. Answer out loud or in the chat.

“A training run is about to start. What three things do you check first?”

Bonus: which chmod number would you use for a .env file, and why?

NEXT · MODULE 3 OF 3

Keep It Alive

From file at rest to process at work: find, watch and stop the processes that serve your model.

Will it fit? · Can it read? · Is it secret?

Emmanuel Naweji · github.com/Here2ServeU/linux-for-ai-ml

APPENDIX A

Exercises: do these before Module 3

01
Free space

df -h ~ How many GB are free?

02
Read the bits

ls -l /etc/passwd /etc/shadow Why can you read one but not the other?

03
Break & fix

touch x; chmod 000 x; cat x then fix with chmod 644 x

04
Octal

Write rwxr-x--- as a number. (750)

05
Hunt secrets

grep -rnE '(hf_|sk-)[A-Za-z0-9]{20,}' ~/ai-job/configs finds nothing after your fix

06
Env vars

export DEMO=hi; echo $DEMO; unset DEMO; echo $DEMO

07
Break & fix again

make_job.py --reset, then reach GO in 5 min

08
Teach it

Explain the three questions to someone in 60 seconds

Glossary

Checkpoint

A save file of a model mid-training.

Scratch / quota

Work space for a job, and the most it may use.

df / du

Free space on a disk · space used by things.

Permission string

-rw-r--r--: type, owner, group, others.

chmod / chown

Change who can do what · change the owner.

644 / 600

Data files · secrets (only you).

Token

A password for a service, e.g. hf_… for Hugging Face.

.env

Hidden file of NAME=value lines. chmod 600. Never commit.

.gitignore

Files Git must never save.

Environment variable

A named value the shell gives every program.

grep / sed

Find text · edit text.

sudo

Run one command as administrator.